# RLC LOW PASS FILTER PROJECT FULL CODE

APPENDICES:

## RLC Low-Pass Filter Numerical Simulation

Course: EEE 415 – Numerical Methods II

Project: Frequency Response Analysis of an RLC Low-Pass Filter

TIMELINE 1

**Defining Circuit Parameters**

```
clc;            % Clear command window

clear;          % Remove variables from workspace

close all;      % Close all open figures

R = 1000;        % Resistance (Ohms)
L = 10e-3;       % Inductance (Henries) -> 10 mH

C = 1e-6;        % Capacitance (Farads) -> 1 uF
```

**Defining Frequency Range for Analysis**

```
f_min = 10;          % Minimum frequency (Hz)

f_max = 1e5;         % Maximum frequency (100 kHz)

N = 1000;            % Number of frequency points

f = linspace(f_min, f_max, N);   % Frequency vector
```

**Converting Frequency to Angular Frequency**

```
w = 2*pi*f;         % Angular frequency (rad/s)

%% 4. Define Laplace Variable s = jω
s = 1j*w;           % j = imaginary unit
disp("Environment and circuit parameters successfully initialized.");
```

TIMELINE 2

### Defining Transfer Function of RLC Low-Pass Filter

```
H = 1 ./ (L*C*s.^2 + R*C*s + 1);    % Transfer function H(s)
```

### Computing Gain in Decibels

```
Gain_dB = 20*log10(abs(H));         % Magnitude in dB
```

### Plotting the Bode Magnitude Plot

```
figure;
semilogx(f, Gain_dB,'LineWidth',2)
grid on
xlabel('Frequency (Hz)')
ylabel('Gain (dB)')
title('Bode Magnitude Plot of RLC Low-Pass Filter')
```

### Saving Figure for Report and Presentation

```
saveas(gcf,'Bode_Magnitude_Plot.png')
disp("Bode plot successfully generated and saved.")
```

## TIMELINE 3

### Forward Difference Numerical Derivative

```
% Step size
h = f(2) - f(1);
% Initialize derivative vector
ForwardDiff = zeros(1,length(Gain_dB));
% Compute Forward Difference

for i = 1:length(Gain_dB)-1

    ForwardDiff(i) = (Gain_dB(i+1) - Gain_dB(i)) / h;

end

% Handle last element
ForwardDiff(end) = ForwardDiff(end-1);
```

### Detecting Numerical Cutoff Frequency

```
[minSlope, idx] = min(ForwardDiff);   % Find steepest slope

fc_forward = f(idx);                  % Cutoff frequency estimate

fprintf('Forward Difference Estimated Cutoff Frequency: %.2f Hz\n', fc_forward);
```

**Plotting Forward Difference Slope**

```
figure



semilogx(f, ForwardDiff,'LineWidth',2)
grid on

hold on

% Mark cutoff frequency on plot
plot(fc_forward, minSlope,'ro','MarkerSize',8,'LineWidth',2)

xlabel('Frequency (Hz)')
ylabel('d(Gain)/df')

title('Forward Difference Approximation of Gain Slope')

legend('Forward Difference Slope','Estimated Cutoff Frequency')
```

**Saving the forward difference Figure**

```
saveas(gcf,'Forward_Difference_Slope.png')

disp("Forward difference slope plotted and cutoff frequency detected.")
```

TIMELINE 4

**Backward Difference Numerical Derivative**

```
% Initialize derivative vector

BackwardDiff = zeros(1,length(Gain_dB));

% Compute Backward Difference
for i = 2:length(Gain_dB)

    BackwardDiff(i) = (Gain_dB(i) - Gain_dB(i-1)) / h;

end

% Handle first element
BackwardDiff(1) = BackwardDiff(2);
```

**Detecting the backward Numerical Cutoff Frequency**

```
 [minSlope_b, idx_b] = min(BackwardDiff);
fc_backward = f(idx_b);

fprintf('Backward Difference Estimated Cutoff Frequency: %.2f Hz\n', fc_backward);
```

**Plotting the Backward Difference Slope**

```
figure
semilogx(f, BackwardDiff,'LineWidth',2)
grid on

hold on

% Mark cutoff point
plot(fc_backward, minSlope_b,'ro','MarkerSize',8,'LineWidth',2)

xlabel('Frequency (Hz)')
ylabel('d(Gain)/df')

title('Backward Difference Approximation of Gain Slope')

legend('Backward Difference Slope','Estimated Cutoff Frequency')
```

**Saving the backward difference Figure**

```
saveas(gcf,'Backward_Difference_Slope.png')
disp("Backward difference slope plotted and saved.")
```

TIMELINE 5

**Central Difference Numerical Derivative**

```
% Initialize derivative vector
CentralDiff = zeros(1,length(Gain_dB));

% Compute Central Difference
for i = 2:length(Gain_dB)-1

    CentralDiff(i) = (Gain_dB(i+1) - Gain_dB(i-1)) / (2*h);

end

% Handle boundary values
CentralDiff(1) = CentralDiff(2);

CentralDiff(end) = CentralDiff(end-1);
```

**Detecting the central difference Numerical Cutoff Frequency**

```
 [minSlope_c, idx_c] = min(CentralDiff);
fc_central = f(idx_c);
```

```
fprintf('Central Difference Estimated Cutoff Frequency: %.2f Hz\n', fc_central);
```

**Plotting Central Difference Slope**

```
figure
semilogx(f, CentralDiff,'LineWidth',2)
grid on

hold on

plot(fc_central, minSlope_c,'ro','MarkerSize',8,'LineWidth',2)
xlabel('Frequency (Hz)')
ylabel('d(Gain)/df')

title('Central Difference Approximation of Gain Slope')

legend('Central Difference','Estimated Cutoff Frequency')
```

**Saving the central difference Figure**

```
saveas(gcf,'Central_Difference_Slope.png')
```

TIMELINE 6

**Step Size Analysis**

```
%% Step Size Analysis for Central Difference Derivative
% Define step sizes (Hz)
step_sizes = [2 5 10];

% Prepare figure
figure;

hold on;

grid on;

colors = {'r', 'g', 'b'};  % line colors for each step size
for k = 1:length(step_sizes)
    h_step = step_sizes(k);

    % Generate frequency vector with this step size

    f_step = f_min:h_step:f_max;

    % Interpolate Gain at these frequency points for smooth calculation

    Gain_step = interp1(f, Gain_dB, f_step, 'linear');



    % Ensure at least 3 points for central difference
```

```
    if length(f_step) < 3

        continue;

    end

    % Compute central difference derivative

    h_vec = diff(f_step);  % spacing between points

    centralDiff = (Gain_step(3:end) - Gain_step(1:end-2)) ./ (2*h_vec(1:end-1));

    % Handle boundaries by repeating nearest slope

    centralDiff = [centralDiff(1), centralDiff, centralDiff(end)];

    % Plot

    semilogx(f_step, centralDiff, 'LineWidth', 2, 'Color', colors{k}, ...

        'DisplayName', ['h = ' num2str(h_step) ' Hz']);

end

% Labels and title
xlabel('Frequency (Hz)');

ylabel('d(Gain)/df');

title('Effect of Step Size on Central Difference');

legend('show', 'FontSize', 8, 'Location', 'best');

hold off;
```

**Saving the Figure for the step size analysis**

```
saveas(gcf,'Step_Size_Analysis.png')
disp("Step size analysis completed and figure saved.")
```

TIMELINE 7

**Computing the Analytical Cutoff Frequency**

```
fc_analytical = 1/(2*pi*sqrt(L*C));
fprintf('Analytical Cutoff Frequency: %.2f Hz\n', fc_analytical);
```

**Creating the Comparison Table**

```
Method = ["Analytical";
          "Forward Difference";

          "Backward Difference";

          "Central Difference"];

Cutoff_Frequency_Hz = [fc_analytical;
                       fc_forward;

                       fc_backward;

                       fc_central];

ComparisonTable = table(Method, Cutoff_Frequency_Hz);
disp("Cutoff Frequency Comparison Table:")
disp(ComparisonTable)
```

**Saving Table to File**

```
writetable(ComparisonTable,'Cutoff_Frequency_Comparison.csv');



disp("Comparison table saved successfully.")
```

## FOR THE MALAB GUI

```
classdef app_for_low_pass_filter < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure          matlab.ui.Figure
        ControlPanel      matlab.ui.container.Panel
        swPlayReplay      matlab.ui.control.Switch
        SwitchLabel       matlab.ui.control.Label
        btnRecordVoice    matlab.ui.control.Button
        btnUploadVoice    matlab.ui.control.Button
        PLOTSPanel        matlab.ui.container.Panel
        btnCompareCutoff  matlab.ui.control.Button
        btnStepSize       matlab.ui.control.Button
        btnPlotBode       matlab.ui.control.Button
        btnForwardDiff    matlab.ui.control.Button
        btnCentralDiff    matlab.ui.control.Button
        btnBackwardDiff   matlab.ui.control.Button
        SOUNDTESTERPanel  matlab.ui.container.Panel
        btnPlayFiltered   matlab.ui.control.Button
        btnPlayOriginal   matlab.ui.control.Button
        PlayCentral       matlab.ui.control.Button
        PlayForward       matlab.ui.control.Button
        PlayBackward      matlab.ui.control.Button
        UIAxes            matlab.ui.control.UIAxes
```

```
        UIAxes2           matlab.ui.control.UIAxes
    end


    properties (Access = public)
    voiceData
    Fs
    filteredSignal
        Gain              % stores gain from Bode plot
    f                 % stores frequency vector
    fc_forward        % forward difference cutoff frequency
    fc_backward       % backward difference cutoff frequency
    fc_central        % central difference cutoff frequency
 % Description
 currentAudio   % stores the audio to play with switch
fsAudio        % sampling rate of the currentAudio
playReady = false  % flag to indicate if switch playback is ready
    end

    properties (Access = private)
    audioData
    fs
   audioPlayer    % global audioplayer object controlling all playback
end



    % Callbacks that handle component events
    methods (Access = private)

        % Button pushed function: btnUploadVoice
        function btnUploadVoiceButtonPushed(app, event)
            % Open a window to select audio file
[file, path] = uigetfile('*.wav','Select Voice File');

% If user cancels
if isequal(file,0)
    return
end

% Read the audio file
[app.voiceData, app.Fs] = audioread(fullfile(path,file));

% Create time axis
t = (0:length(app.voiceData)-1)/app.Fs;  % time vector
plot(app.UIAxes, t, app.voiceData)

xlabel(app.UIAxes, 'Time (seconds)')
ylabel(app.UIAxes, 'Amplitude')
title(app.UIAxes, 'Original Voice Signal')

grid(app.UIAxes, 'on')
[app.voiceData, app.Fs] = audioread(fullfile(path,file));

% Add these two lines:
```

```
app.audioData = app.voiceData;
app.fs = app.Fs;
        end

        % Button pushed function: btnPlayFiltered
        function btnPlayFilteredButtonPushed(app, event)

         % Stop any currently playing audio
if ~isempty(app.audioPlayer) && isvalid(app.audioPlayer) &&
isplaying(app.audioPlayer)
    stop(app.audioPlayer);
end
    % Filter parameters
    fc = 1000;  % cutoff frequency
    order = 4;

    % Design Butterworth low-pass filter
    [b, a] = butter(order, fc/(app.Fs/2));

    % Apply filter
    app.filteredSignal = filter(b, a, app.voiceData);

    % Plot filtered waveform
    t = (0:length(app.filteredSignal)-1)/app.Fs;
    plot(app.UIAxes2, t, app.filteredSignal)
    xlabel(app.UIAxes2, 'Time (seconds)')
    ylabel(app.UIAxes2, 'Amplitude')
    title(app.UIAxes2, 'Filtered Voice Signal')
    grid(app.UIAxes2, 'on')



    app.audioPlayer = audioplayer(app.filteredSignal, app.Fs);
play(app.audioPlayer);

app.currentAudio = app.filteredSignal;
app.fsAudio = app.Fs;
app.playReady = true;
app.swPlayReplay.Value = 'Off'; % reset switch

        end

        % Button pushed function: btnPlotBode
        function btnPlotBodeButtonPushed(app, event)


    % Circuit parameters
    R = 1000;
    L = 10e-3;
    C = 1e-6;

    f = logspace(1,5,1000);    % frequency vector
    w = 2*pi*f;
    s = 1j*w;
```

```
    % Transfer function
    H = 1 ./ (L*C*s.^2 + R*C*s + 1);
    Gain = 20*log10(abs(H));

    % Save Gain for other buttons
    app.Gain = Gain;   % store in app property
    app.f = f;         % store frequency vector too

    % Plot
    semilogx(app.UIAxes2, f, Gain, 'LineWidth', 2);
    grid(app.UIAxes2, 'on');
    xlabel(app.UIAxes2, 'Frequency (Hz)');
    ylabel(app.UIAxes2, 'Gain (dB)');
    title(app.UIAxes2, 'Bode Magnitude Plot');

        end

        % Button pushed function: btnForwardDiff
        function btnForwardDiffButtonPushed(app, event)

  if isempty(app.f) || isempty(app.Gain)
        uialert(app.UIFigure, 'Please click Plot Bode first.', 'Missing Data');
        return
    end

    h = diff(app.f);
    forwardDiff = (app.Gain(2:end) - app.Gain(1:end-1)) ./ h;

    [~, idx] = min(forwardDiff);
    app.fc_forward = app.f(idx);

    cla(app.UIAxes2)
    plot(app.UIAxes2, app.f(1:end-1), forwardDiff, 'LineWidth', 2)
    grid(app.UIAxes2, 'on')
    hold(app.UIAxes2, 'on')
    xline(app.UIAxes2, app.fc_forward, 'g--', 'Forward Cutoff')
    xlabel(app.UIAxes2, 'Frequency (Hz)')
    ylabel(app.UIAxes2, 'd(Gain)/df')
    title(app.UIAxes2, 'Forward Difference Gain Slope')
        end

        % Button pushed function: btnBackwardDiff
        function btnBackwardDiffButtonPushed(app, event)
    cla(app.UIAxes2)
hold(app.UIAxes2, 'off')

  h = diff(app.f);
backwardDiff = (app.Gain(2:end) - app.Gain(1:end-1)) ./ h;

[~, idx] = min(backwardDiff);
app.fc_backward = app.f(idx+1);  % shift because backward uses previous point

plot(app.UIAxes2, app.f(2:end), backwardDiff, 'LineWidth', 2)
grid(app.UIAxes2, 'on')
hold(app.UIAxes2, 'on')
```

```
xline(app.UIAxes2, app.fc_backward, 'r--', 'Backward Cutoff')
xlabel(app.UIAxes2, 'Frequency (Hz)')
ylabel(app.UIAxes2, 'd(Gain)/df')
title(app.UIAxes2, 'Backward Difference Gain Slope')
        end

        % Button pushed function: btnCentralDiff
        function btnCentralDiffButtonPushed(app, event)
    cla(app.UIAxes2)
hold(app.UIAxes2, 'off')

    h = diff(app.f);
centralDiff = (app.Gain(3:end) - app.Gain(1:end-2)) ./ (2*h(1:end-1));

[~, idx] = min(centralDiff);
app.fc_central = app.f(idx+1);  % shift index because of central diff

plot(app.UIAxes2, app.f(2:end-1), centralDiff, 'LineWidth', 2)
grid(app.UIAxes2, 'on')
hold(app.UIAxes2, 'on')
xline(app.UIAxes2, app.fc_central, 'b--', 'Central Cutoff')
xlabel(app.UIAxes2, 'Frequency (Hz)')
ylabel(app.UIAxes2, 'd(Gain)/df')
title(app.UIAxes2, 'Central Difference Gain Slope')
        end

        % Button pushed function: btnStepSize
        function btnStepSizeButtonPushed(app, event)

    if isempty(app.Gain) || isempty(app.f)
        uialert(app.UIFigure, 'Please plot the Bode magnitude first.', 'Error');
        return
    end

    % Original frequency vector and Gain
    f_orig = app.f;
    Gain_orig = app.Gain;

    % Different step sizes (take every nth point)
    stepSizes = [2, 5, 10];  % smaller steps give higher resolution

    % Clear previous plot
    cla(app.UIAxes2)
    hold(app.UIAxes2, 'on')

    colors = {'r','g','b'}; % colors for different step sizes

    for i = 1:length(stepSizes)
        % Reduce frequency points
        f_step = f_orig(1:stepSizes(i):end);
        Gain_step = Gain_orig(1:stepSizes(i):end);

        % Compute central difference for reduced points
        h = diff(f_step);
        centralDiff = (Gain_step(3:end) - Gain_step(1:end-2)) ./ (2*h(1:end-1));
```

```
        % Plot
        plot(app.UIAxes2, f_step(2:end-1), centralDiff, 'LineWidth', 2, ...
            'Color', colors{i}, 'DisplayName', ['Step = ' num2str(stepSizes(i))])
    end

    % Labels and title
    xlabel(app.UIAxes2, 'Frequency (Hz)')
    ylabel(app.UIAxes2, 'd(Gain)/df')
    title(app.UIAxes2, 'Step Size Effect on Central Difference')
    grid(app.UIAxes2, 'on')

    % Small legend
    legend(app.UIAxes2, 'show', 'FontSize', 8, 'Location', 'best')
        end

        % Button pushed function: btnCompareCutoff
        function btnCompareCutoffButtonPushed(app, event)


% Ensure Gain and frequency vectors exist
    if isempty(app.Gain) || isempty(app.f)
        uialert(app.UIFigure, 'Please plot the Bode magnitude first.', 'Error');
        return
    end

    f = app.f;
    Gain = app.Gain;

    % --- Forward Difference ---
    h_forward = diff(f);
    forwardDiff = (Gain(2:end) - Gain(1:end-1)) ./ h_forward;
    [~, idx_f] = min(forwardDiff);
    fc_forward = f(idx_f);

    % --- Backward Difference ---
    h_backward = diff(f);
    backwardDiff = (Gain(2:end) - Gain(1:end-1)) ./ h_backward;
    [~, idx_b] = min(backwardDiff);
    fc_backward = f(idx_b);

    % --- Central Difference ---
    h_central = diff(f);
    centralDiff = (Gain(3:end) - Gain(1:end-2)) ./ (2*h_central(1:end-1));
    [~, idx_c] = min(centralDiff);
    fc_central = f(idx_c + 1); % shift index

    % Store cutoff frequencies in app properties
    app.fc_forward = fc_forward;
    app.fc_backward = fc_backward;
    app.fc_central = fc_central;

    % --- Plot all derivatives ---
    cla(app.UIAxes2)
    hold(app.UIAxes2,'on')
```

```
    % Forward
    plot(app.UIAxes2, f(1:end-1), forwardDiff, 'g', 'LineWidth', 1.5, 'DisplayName',
'Forward Diff');
    plot(app.UIAxes2, fc_forward, forwardDiff(idx_f), 'go',
'MarkerFaceColor','g','MarkerSize',6)

    % Backward
    plot(app.UIAxes2, f(1:end-1), backwardDiff, 'r', 'LineWidth', 1.5, 'DisplayName',
'Backward Diff');
    plot(app.UIAxes2, fc_backward, backwardDiff(idx_b), 'ro',
'MarkerFaceColor','r','MarkerSize',6)

    % Central
    plot(app.UIAxes2, f(2:end-1), centralDiff, 'b', 'LineWidth', 2, 'DisplayName',
'Central Diff');
    plot(app.UIAxes2, fc_central, centralDiff(idx_c), 'bo',
'MarkerFaceColor','b','MarkerSize',6)

    % Labels and grid
    xlabel(app.UIAxes2,'Frequency (Hz)')
    ylabel(app.UIAxes2,'d(Gain)/df')
    title(app.UIAxes2,'Comparison of Cutoff Frequencies')
    grid(app.UIAxes2,'on')

    % Small legend placed above the plot

legend(app.UIAxes2,'show','FontSize',7,'Location','northoutside','Orientation','horiz
ontal')

    % --- Zoom around steepest slope (Central Difference) ---
    zoomRange = 5; % number of points around central cutoff
    idx_start = max(idx_c - zoomRange, 1);
    idx_end = min(idx_c + zoomRange, length(f)-2);
    xlim(app.UIAxes2, [f(idx_start) f(idx_end)])
        end

        % Button pushed function: btnPlayOriginal
        function btnPlayOriginalButtonPushed(app, event)
     % Stop any currently playing audio
if ~isempty(app.audioPlayer) && isvalid(app.audioPlayer) &&
isplaying(app.audioPlayer)
    stop(app.audioPlayer);
end

% Ensure audio exists
if isempty(app.voiceData) || isempty(app.Fs)
    uialert(app.UIFigure, 'Please upload a voice file first.', 'No Audio');
    return
end

% Plot the original waveform again (optional)
t = (0:length(app.voiceData)-1)/app.Fs;  % time vector
plot(app.UIAxes, t, app.voiceData, 'b', 'LineWidth', 1.5)
xlabel(app.UIAxes, 'Time (seconds)')
```

```
ylabel(app.UIAxes, 'Amplitude')
title(app.UIAxes, 'Original Voice Signal')
grid(app.UIAxes, 'on')

% Play the original audio using the shared audioplayer
app.audioPlayer = audioplayer(app.voiceData, app.Fs);
play(app.audioPlayer);

app.currentAudio = app.filteredSignal;
app.fsAudio = app.Fs;
app.playReady = true;
app.swPlayReplay.Value = 'Off'; % reset switch
        end

        % Button pushed function: btnRecordVoice
        function btnRecordVoiceButtonPushed(app, event)

    Fs = 44100;          % Sampling frequency (Hz)
    nBits = 16;          % Bits per sample
    nChannels = 1;       % Mono recording
    recTime = 10;        % Recording time in seconds

    % Create recorder object
    recorder = audiorecorder(Fs, nBits, nChannels);

    % Inform user
    uialert(app.UIFigure, ['Recording for ' num2str(recTime) ' seconds.'], 'Recording
Started');

    % Record audio (blocking)
    recordblocking(recorder, recTime);

    % Retrieve recorded audio
    app.voiceData = getaudiodata(recorder);
    app.Fs = Fs;

    % Plot the recorded waveform
    t = (0:length(app.voiceData)-1)/app.Fs;
    plot(app.UIAxes, t, app.voiceData, 'r', 'LineWidth', 1.5)
    xlabel(app.UIAxes, 'Time (seconds)')
    ylabel(app.UIAxes, 'Amplitude')
    title(app.UIAxes, 'Recorded Voice Signal')
    grid(app.UIAxes, 'on')

    % Inform user recording finished
    uialert(app.UIFigure, 'Recording finished.', 'Done');

    app.voiceData = getaudiodata(recorder);
app.Fs = Fs;

% Add these two lines:
app.audioData = app.voiceData;
app.fs = app.Fs;
        end
```

```
        % Button pushed function: PlayForward
        function PlayForwardButtonPushed(app, event)
          % --- Stop any currently playing audio ---
if ~isempty(app.audioPlayer) && isvalid(app.audioPlayer) &&
isplaying(app.audioPlayer)
    stop(app.audioPlayer);
end

% --- Check if audio is loaded ---
if isempty(app.audioData) || isempty(app.fs)
    uialert(app.UIFigure, 'Please upload or record audio first.', 'Missing Audio');
    return
end

% --- Get audio ---
x = app.audioData;

% --- Convert stereo to mono if necessary ---
if size(x,2) == 2
    x = mean(x,2);
end

% --- Apply forward difference ---
y_forward = diff(x);

% --- Normalize to avoid clipping ---
if max(abs(y_forward)) ~= 0
    y_forward = y_forward / max(abs(y_forward));
end

% --- Plot waveform on second axes (UIAxes2) ---
t = (0:length(y_forward)-1)/app.fs;  % time vector
plot(app.UIAxes2, t, y_forward, 'b', 'LineWidth', 1.5)
xlabel(app.UIAxes2, 'Time (s)')
ylabel(app.UIAxes2, 'Amplitude')
title(app.UIAxes2, 'Forward Difference Audio Waveform')
grid(app.UIAxes2, 'on')

% --- Set current audio for switch control ---
app.currentAudio = y_forward;   % correct variable for forward audio
app.fsAudio = app.fs;
app.playReady = true;

% Optional: reset switch to OFF
app.swPlayReplay.Value = 'Off';  % your switch name here

% --- Create and play audioplayer ---
app.audioPlayer = audioplayer(y_forward, app.fs);
play(app.audioPlayer);

        end

        % Button pushed function: PlayBackward
        function PlayBackwardButtonPushed(app, event)
% --- Stop any currently playing audio ---
```

```
if ~isempty(app.audioPlayer) && isvalid(app.audioPlayer) &&
isplaying(app.audioPlayer)
    stop(app.audioPlayer);
end

% --- Check if audio is loaded ---
if isempty(app.audioData) || isempty(app.fs)
    uialert(app.UIFigure, 'Please upload or record audio first.', 'Missing Audio');
    return
end

% --- Get audio ---
x = app.audioData;

% --- Convert stereo to mono if necessary ---
if size(x,2) == 2
    x = mean(x,2);
end

% --- Apply backward difference ---
% y[n] = x[n] - x[n-1], prepend 0 to match length
y_backward = [0; diff(x)];

% --- Normalize to avoid clipping ---
if max(abs(y_backward)) ~= 0
    y_backward = y_backward / max(abs(y_backward));
end

% --- Plot waveform on UIAxes2 ---
t = (0:length(y_backward)-1)/app.fs;  % time vector
plot(app.UIAxes2, t, y_backward, 'r', 'LineWidth', 1.5)
xlabel(app.UIAxes2, 'Time (s)')
ylabel(app.UIAxes2, 'Amplitude')
title(app.UIAxes2, 'Backward Difference Audio Waveform')
grid(app.UIAxes2, 'on')

% --- Set current audio for switch control ---
app.currentAudio = y_backward;   % correct variable for backward
app.fsAudio = app.fs;
app.playReady = true;

% Optional: reset switch to OFF
app.swPlayReplay.Value = 'Off';  % your switch name here

% --- Create and play audioplayer ---
app.audioPlayer = audioplayer(y_backward, app.fs);
play(app.audioPlayer);
        end

        % Button pushed function: PlayCentral
        function PlayCentralButtonPushed(app, event)
         % Stop any currently playing audio
if ~isempty(app.audioPlayer) && isvalid(app.audioPlayer) &&
isplaying(app.audioPlayer)
    stop(app.audioPlayer);
```

```
end
            % --- Check if audio is loaded ---
    if isempty(app.audioData) || isempty(app.fs)
        uialert(app.UIFigure, 'Please upload or record audio first.', 'Missing
Audio');
        return
    end

    % --- Get audio ---
    x = app.audioData;

    % --- Convert stereo to mono if necessary ---
    if size(x,2) == 2
        x = mean(x,2);
    end

    % --- Apply central difference ---
    % y[n] = (x[n+1] - x[n-1])/2
    y_central = zeros(size(x));
    y_central(2:end-1) = (x(3:end) - x(1:end-2))/2;

    % --- Normalize to avoid clipping ---
    if max(abs(y_central)) ~= 0
        y_central = y_central / max(abs(y_central));
    end

    % --- Plot waveform on UIAxes2 ---
    t = (0:length(y_central)-1)/app.fs;  % time vector
    plot(app.UIAxes2, t, y_central, 'b', 'LineWidth', 1.5)
    xlabel(app.UIAxes2, 'Time (s)')
    ylabel(app.UIAxes2, 'Amplitude')
    title(app.UIAxes2, 'Central Difference Audio Waveform')
    grid(app.UIAxes2, 'on')

% --- Update audioplayer line ---
app.audioPlayer = audioplayer(y_central, app.fs);
play(app.audioPlayer);

% --- Correct current audio assignment ---
app.currentAudio = y_central;
app.fsAudio = app.fs;
app.playReady = true;

        end

        % Value changed function: swPlayReplay
        function swPlayReplayValueChanged(app, event)

    value = app.swPlayReplay.Value;

    if ~app.playReady
        uialert(app.UIFigure, 'Please press Forward, Backward, Central, Original, or
Filtered first.', 'No Audio Selected');
        app.swPlayReplay.Value = 'Off';
        return
```

```
    end

    switch value
        case 'On'
            if isempty(app.audioPlayer) || ~isvalid(app.audioPlayer)
                app.audioPlayer = audioplayer(app.currentAudio, app.fsAudio);
            end
            play(app.audioPlayer);
        case 'Off'
            if ~isempty(app.audioPlayer) && isplaying(app.audioPlayer)
                stop(app.audioPlayer);
            end
    end
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.Position = [100 100 658 480];
            app.UIFigure.Name = 'MATLAB App';

            % Create UIAxes2
            app.UIAxes2 = uiaxes(app.UIFigure);
            title(app.UIAxes2, 'Numerical Plots')
            xlabel(app.UIAxes2, 'Time / Frequency (Hz)')
            ylabel(app.UIAxes2, 'Amplitude / Gain (dB)')
            zlabel(app.UIAxes2, 'Z')
            app.UIAxes2.Position = [341 289 257 162];

            % Create UIAxes
            app.UIAxes = uiaxes(app.UIFigure);
            title(app.UIAxes, 'Voice Waveform')
            xlabel(app.UIAxes, 'Time / Frequency (Hz)')
            ylabel(app.UIAxes, 'Amplitude / Gain (dB)')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.Position = [57 289 230 161];

            % Create SOUNDTESTERPanel
            app.SOUNDTESTERPanel = uipanel(app.UIFigure);
            app.SOUNDTESTERPanel.Title = 'SOUND TESTER';
            app.SOUNDTESTERPanel.Position = [362 129 260 142];

            % Create PlayBackward
            app.PlayBackward = uibutton(app.SOUNDTESTERPanel, 'push');
            app.PlayBackward.ButtonPushedFcn = createCallbackFcn(app,
@PlayBackwardButtonPushed, true);
            app.PlayBackward.Position = [12 15 94 23];
            app.PlayBackward.Text = 'Play Backward';
```

```
            % Create PlayForward
            app.PlayForward = uibutton(app.SOUNDTESTERPanel, 'push');
            app.PlayForward.ButtonPushedFcn = createCallbackFcn(app,
@PlayForwardButtonPushed, true);
            app.PlayForward.Position = [148 15 89 23];
            app.PlayForward.Text = 'Play Forward ';

            % Create PlayCentral
            app.PlayCentral = uibutton(app.SOUNDTESTERPanel, 'push');
            app.PlayCentral.ButtonPushedFcn = createCallbackFcn(app,
@PlayCentralButtonPushed, true);
            app.PlayCentral.Position = [80 54 100 23];
            app.PlayCentral.Text = 'Play Central';

            % Create btnPlayOriginal
            app.btnPlayOriginal = uibutton(app.SOUNDTESTERPanel, 'push');
            app.btnPlayOriginal.ButtonPushedFcn = createCallbackFcn(app,
@btnPlayOriginalButtonPushed, true);
            app.btnPlayOriginal.Position = [17 85 83 23];
            app.btnPlayOriginal.Text = 'Play Original';

            % Create btnPlayFiltered
            app.btnPlayFiltered = uibutton(app.SOUNDTESTERPanel, 'push');
            app.btnPlayFiltered.ButtonPushedFcn = createCallbackFcn(app,
@btnPlayFilteredButtonPushed, true);
            app.btnPlayFiltered.Position = [159 85 82 23];
            app.btnPlayFiltered.Text = 'Play Filtered';

            % Create PLOTSPanel
            app.PLOTSPanel = uipanel(app.UIFigure);
            app.PLOTSPanel.Title = '
PLOTS';
            app.PLOTSPanel.Position = [57 16 565 99];

            % Create btnBackwardDiff
            app.btnBackwardDiff = uibutton(app.PLOTSPanel, 'push');
            app.btnBackwardDiff.ButtonPushedFcn = createCallbackFcn(app,
@btnBackwardDiffButtonPushed, true);
            app.btnBackwardDiff.Position = [164 51 121 21];
            app.btnBackwardDiff.Text = 'Backward Difference';

            % Create btnCentralDiff
            app.btnCentralDiff = uibutton(app.PLOTSPanel, 'push');
            app.btnCentralDiff.ButtonPushedFcn = createCallbackFcn(app,
@btnCentralDiffButtonPushed, true);
            app.btnCentralDiff.Position = [317 50 112 23];
            app.btnCentralDiff.Text = 'Central Difference';

            % Create btnForwardDiff
            app.btnForwardDiff = uibutton(app.PLOTSPanel, 'push');
            app.btnForwardDiff.ButtonPushedFcn = createCallbackFcn(app,
@btnForwardDiffButtonPushed, true);
            app.btnForwardDiff.Position = [18 51 107 20];
            app.btnForwardDiff.Text = 'Forward Difference';
```

```
            % Create btnPlotBode
            app.btnPlotBode = uibutton(app.PLOTSPanel, 'push');
            app.btnPlotBode.ButtonPushedFcn = createCallbackFcn(app,
@btnPlotBodeButtonPushed, true);
            app.btnPlotBode.Position = [464 50 67 23];
            app.btnPlotBode.Text = 'Plot Bode';

            % Create btnStepSize
            app.btnStepSize = uibutton(app.PLOTSPanel, 'push');
            app.btnStepSize.ButtonPushedFcn = createCallbackFcn(app,
@btnStepSizeButtonPushed, true);
            app.btnStepSize.Position = [169 11 112 23];
            app.btnStepSize.Text = 'Step size Analysis';

            % Create btnCompareCutoff
            app.btnCompareCutoff = uibutton(app.PLOTSPanel, 'push');
            app.btnCompareCutoff.ButtonPushedFcn = createCallbackFcn(app,
@btnCompareCutoffButtonPushed, true);
            app.btnCompareCutoff.Position = [322 11 97 23];
            app.btnCompareCutoff.Text = 'Compare cutoff';

            % Create ControlPanel
            app.ControlPanel = uipanel(app.UIFigure);
            app.ControlPanel.Title = 'Control Panel';
            app.ControlPanel.Position = [57 129 260 142];

            % Create btnUploadVoice
            app.btnUploadVoice = uibutton(app.ControlPanel, 'push');
            app.btnUploadVoice.ButtonPushedFcn = createCallbackFcn(app,
@btnUploadVoiceButtonPushed, true);
            app.btnUploadVoice.Position = [18 83 80 26];
            app.btnUploadVoice.Text = 'Upload Voice';

            % Create btnRecordVoice
            app.btnRecordVoice = uibutton(app.ControlPanel, 'push');
            app.btnRecordVoice.ButtonPushedFcn = createCallbackFcn(app,
@btnRecordVoiceButtonPushed, true);
            app.btnRecordVoice.Position = [155 82 91 28];
            app.btnRecordVoice.Text = 'Record Voice';

            % Create SwitchLabel
            app.SwitchLabel = uilabel(app.ControlPanel);
            app.SwitchLabel.HorizontalAlignment = 'center';
            app.SwitchLabel.Position = [110 15 40 22];
            app.SwitchLabel.Text = 'Switch';

            % Create swPlayReplay
            app.swPlayReplay = uiswitch(app.ControlPanel, 'slider');
            app.swPlayReplay.ValueChangedFcn = createCallbackFcn(app,
@swPlayReplayValueChanged, true);
            app.swPlayReplay.Position = [107 52 45 20];

            % Show the figure after all components are created
            app.UIFigure.Visible = 'on';
        end
```

```
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = app_for_low_pass_filter

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```